САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №1

«Высокопроизводительные вычисления в механике»

Выполнил: студент группы 4446-010303D

Погудин М.Э.

Проверил: Неженский М.С.

«____» _____2026 г.

Оценка _____

Самара 2026

Для выполнения работы используем компилятор g++. Настройка переменных среды приведена на рисунке 1.

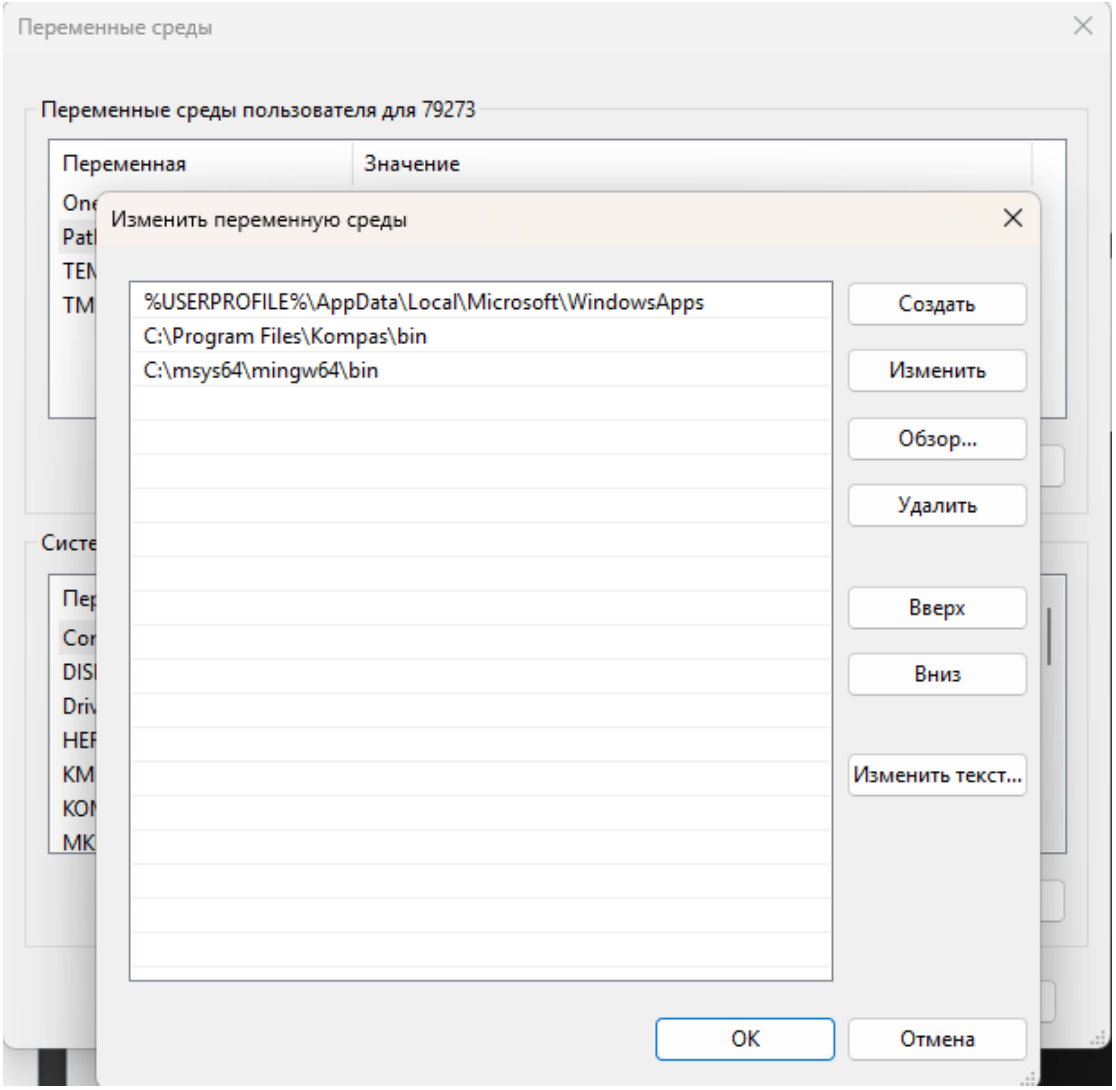


Рисунок 1 – Настройка переменных среды

Скомпилируем приведенный файл. Ниже приведен вывод результат в терминале.

Проведем расчет для n от 500 до 1000, где n - размер матриц. Результаты вычисление приведены на рисунке 2.

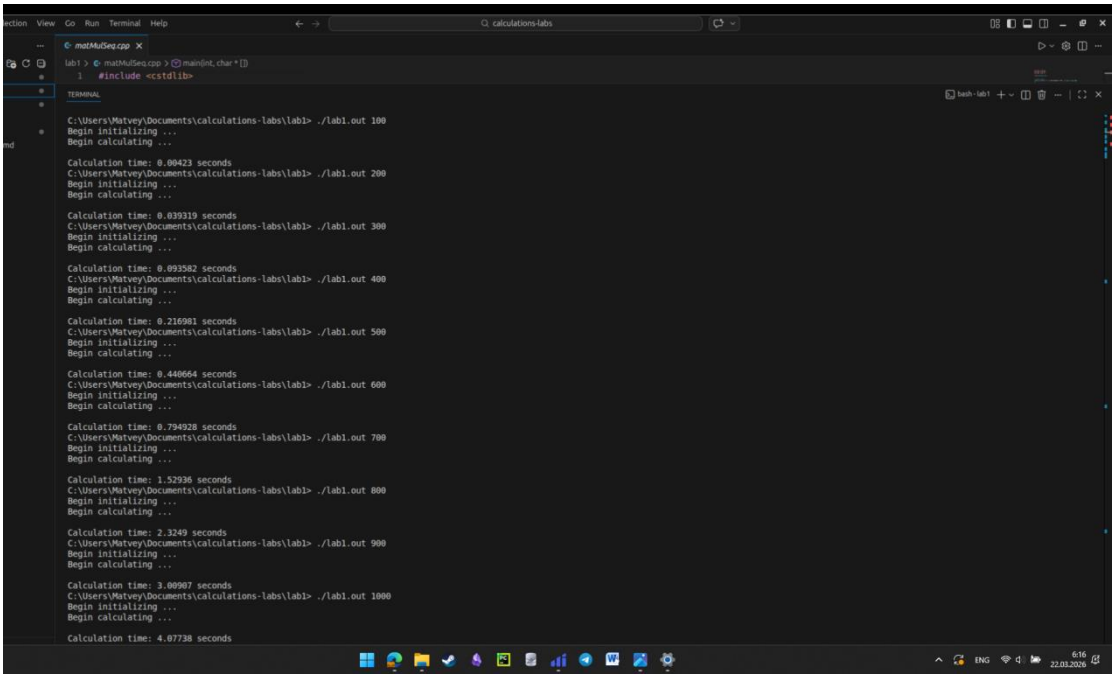


Рисунок 2 - Результаты вычислений

По полученным результатам построим таблицу (см. Таблица 1) и построим график (см. Рисунок 3).

Таблица 1 – Результаты вычислений

| Размер матриц n×n | Время расчета, с |
|---|---|
| 100 | 0,00423 |
| 200 | 0,039319 |
| 300 | 0,093582 |
| 400 | 0,216981 |
| 500 | 0,440664 |
| 600 | 0,794928 |
| 700 | 1,52936 |
| 800 | 2,3249 |
| 900 | 3,00907 |
| 1000 | 4,07738 |

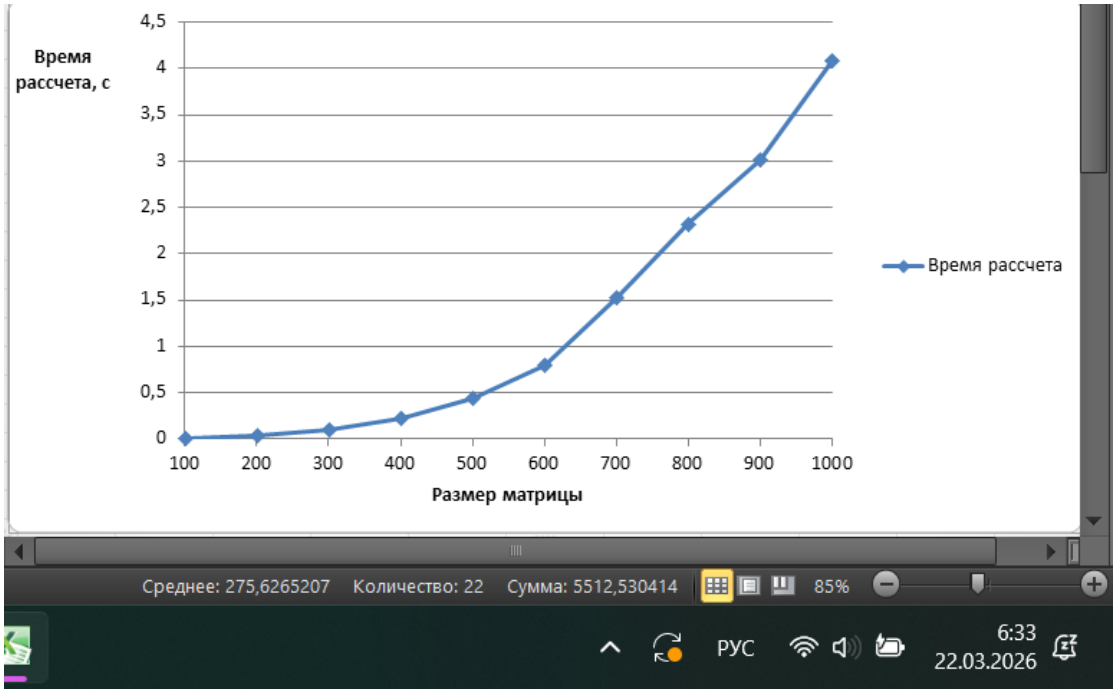


Рисунок 3 – Результаты вычислений

```
//Подключение библиотек
#include <cstdlib>
#include <iostream>
#include "time.h"

using namespace std;


// Функция вывода матрицы
template <typename T> int matrixOutput(T *Mat, int size, string name)
{
    cout << "\"" << name << "\" matrix output:" << endl;
    for (int i=0; i < size; i++)
    {
    for (int j = 0; j < size; j++)
    {
        cout << Mat[i*size + j] << " ";
    }
    cout << endl;
    }

    return 1;
}

int main(int argc, char *argv[])
{
    clock_t start;
    if (argc != 2)
    {
    cout << "Program usage: " << endl <<"./" << argv[0] << " <n>" << endl << "where <n> is the size of square matrix" << endl;
    return -1;
    }

    const int N = atoi(argv[1]);

    cout << "Begin initializing ..." << endl;

    float *A, *B;

    // Выделение памяти для двух новых матриц
    A = new float[N*N];
    B = new float[N*N];

    // Заполнение матриц рандомными числами
    for (int i=0; i < N; i++)
    for (int j = 0; j < N; j++)
    {
        A[i*N + j] = (i+1) * (j+1);
        B[i*N + j] = (i+1) + 2*(j+1);
    }

    //Создание матрицы результата

    float *C = new float[N*N];

    cout << "Begin calculating ..." << endl;

    //Начало отсчета времени
    start = clock();
```

Рисунок 4 – Подробное описание кода (часть 1)

```
    //Начало отсчета времени
    start = clock();

    // Умножение матриц
    for (int i=0; i < N; i++)
    for (int j = 0; j < N; j++)
        {
        C[i*N + j] = 0;
        for (int k = 0; k < N; k++)
        {
        C[i*N + j] += A[i*N + k] * B[k*N + j];
        }
        }

    //Вывод времени рассчета
    cout << endl << "Calculation time: " << double(clock() - start)/CLOCKS_PER_SEC << " seconds" << endl;


    // Очистка памяти
    delete [] A;
    delete [] B;
    delete [] C;

    return 0;
}
```

Рисунок 5 – Подробное описание кода (часть 2)

## Вывод

Сложность рассматриваемого алгоритма матричного умножения составляет $O(n^3)$. Это означает, что время выполнения растёт пропорционально кубу размера входных данных. Следовательно, увеличение размера матриц приводит к заметному увеличению времени вычислений, что полностью соответствует теоретическим ожиданиям для алгоритма с кубической сложностью.

Ссылка на репозиторий с выполненными лабораторными работами:
https://github.com/MatveyPogudin/high-performance-computing